

RAPPORT DE PROJET DE FIN DE MODULE

Conception, Implémentation et Analyse de Modèles de Deep Learning

MLP, CNN et architectures séquentielles (RNN/LSTM/GRU/Seq2Seq)

Réalisé par :

Wiam RAOUB

Module :

Deep Learning

Prof :

Mosaab BATAL

Filière : Intelligence Artificielle & Data Science — 4^{ème} Année

Année Universitaire : 2025/2026

Résumé

Ce projet de fin de module porte sur la conception, l'implémentation et l'analyse critique de trois familles de modèles de **deep learning**, chacune adaptée à un type de données différent : un **perceptron multicouche (MLP)** pour des données tabulaires (Breast Cancer Wisconsin), un **réseau de neurones convolutif (CNN)** de type LeNet pour la classification d'images (Fashion-MNIST), et des architectures **récurrentes (RNN, LSTM, GRU)** ainsi qu'un système **encodeur-décodeur (Seq2Seq)** pour la modélisation de séquences (IMDb et un corpus de traduction simplifié).

L'ensemble du travail a été réalisé sous **PyTorch**, en couvrant à chaque étape la justification théorique des choix d'architecture, l'implémentation, l'expérimentation comparative et l'interprétation critique des résultats obtenus.

Mots-clés : Deep Learning, PyTorch, MLP, CNN, LeNet, RNN, LSTM, GRU, Seq2Seq, BLEU.

Table des matières

Résumé	i
Liste des figures	iv
Liste des tableaux	v
Introduction Générale	1
1 MLP et Ingénierie PyTorch – Classification Tabulaire	2
1.1 Introduction et objectifs	2
1.2 Concepts fondamentaux de PyTorch	2
1.3 Méthodologie et préparation des données	3
1.4 Étude comparative des stratégies d’initialisation	3
1.5 Analyse critique des stratégies d’initialisation	4
1.6 Question de synthèse – Partie I	5
1.7 Concepts fondamentaux de PyTorch	5
2 CNN et Vision par Ordinateur – Classification d’Images	6
2.1 Pourquoi le MLP est inadapté aux images ?	6
2.2 Idées fondatrices des CNN	6
2.3 Corrélation croisée 2D et calculs de dimensions	7
2.4 Rôle de la convolution 1×1	7
2.5 Analyse des choix architecturaux du CNN	8
2.6 Question de synthèse – Partie II	9
3 RNN, LSTM, GRU et Seq2Seq – Modélisation de Séquences	11
3.1 Concepts théoriques des séquences	11
3.2 Effet du Gradient Clipping	12
3.3 Question de synthèse – Partie III	13
4 Discussion Transversale et Conclusion	14
4.1 Question transversale finale	14
4.1.1 Le paradigme commun : apprentissage par gradient	14
4.1.2 MLP : aucune hypothèse sur la structure des données	14
4.1.3 CNN : exploiter la géométrie spatiale	14

4.1.4	RNN / LSTM / GRU : modéliser la dépendance temporelle . . .	15
4.1.5	Tableau synthétique comparatif	15
4.2	Bilan et perspectives	15
4.3	Conclusion Générale	16
Bibliographie		17

Table des figures

1.1	Courbes d'apprentissage (Loss) du MLP selon la méthode d'initialisation	4
1.2	Matrice de confusion finale du modèle MLP sur l'ensemble de validation .	4
2.1	Échantillon d'images du dataset Fashion-MNIST avec leurs classes	7
2.2	Courbe de perte et précision lors de l'entraînement de LeNet-5	8
2.3	Matrice de confusion sur Fashion-MNIST	8
2.4	Visualisation des cartes d'activation (Feature Maps) de la première couche	9
3.1	Comparaison des courbes de perte RNN / LSTM / GRU	11
3.2	Effet du gradient clipping sur la norme des gradients RNN	12
3.3	Seq2Seq training loss	12
3.4	Diagramme comparatif des accuracies RNN / LSTM / GRU	12

Liste des tableaux

2.1	Principes fondateurs des architectures CNN	6
4.1	Comparaison transversale des paradigmes architecturaux	15

Introduction Générale

Le deep learning ne propose pas une architecture universelle : le choix d'un modèle dépend fondamentalement de la nature des données à traiter – tabulaires, spatiales (images) ou séquentielles (texte). Ce projet a pour objectif de démontrer, à travers trois cas d'usage réels, comment cette adaptation se traduit concrètement en termes de théorie, d'implémentation et de résultats expérimentaux.

La problématique transversale qui guide ce travail est la suivante :

Comment le deep learning adapte-t-il ses architectures à la structure des données – tabulaire, image et séquentielle – et pourquoi un même paradigme d'apprentissage supervisé doit-il être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données ?

Pour y répondre, ce rapport est organisé en trois parties correspondant aux trois familles de modèles étudiées, suivies d'une synthèse comparative transversale venant mettre en perspective les liens conceptuels entre les trois approches.

Chapitre 1

MLP et Ingénierie PyTorch – Classification Tabulaire

1.1 Introduction et objectifs

Cette première partie porte sur la classification supervisée de données tabulaires réelles à l'aide d'un perceptron multicouche (MLP). L'objectif est de maîtriser l'ingénierie PyTorch (`nn.Module`, gestion des paramètres, initialisation, sauvegarde/rechargement, device) et d'évaluer la pertinence d'un MLP sur ce type de données.

1.2 Concepts fondamentaux de PyTorch

- **`nn.Module`** : Il s'agit de la classe de base de tout réseau de neurones sous PyTorch [1]. Elle gère automatiquement l'enregistrement des sous-modules et des paramètres, ce qui permet d'appeler `.parameters()`, `.to(device)` ou `.state_dict()` sur un réseau entier sans avoir à le faire manuellement couche par couche.
- **Paramètres et gradients** : Chaque couche `nn.Linear` possède une matrice de poids *weight* et un vecteur *bias*, tous deux des `Parameter` (sous-classe de `Tensor` avec `requires_grad=True`). Lors de l'appel à `loss.backward()`, PyTorch construit dynamiquement le graphe de calcul et remonte le gradient de la perte par rapport à chaque paramètre via la règle de la chaîne (rétropropagation). L'optimiseur (Adam ici) utilise ensuite ces gradients pour mettre à jour les poids : `optimizer.step()`.
- **`state_dict`** : C'est un dictionnaire ordonné qui associe à chaque paramètre (et buffer) son nom et son tenseur de valeurs actuelles. Il est indépendant de l'architecture exacte de la classe Python et c'est ce qui permet de sauvegarder/recharger un modèle (`torch.save` / `load_state_dict`), à condition que l'architecture cible soit identique.
- **Device (CPU/GPU)** : Un tenseur et un modèle doivent résider sur le même device pour qu'une opération soit possible. `torch.device("cuda" if torch.cuda.is_available() else "cpu")`

- Propagation avant (forward) : Pour une entrée tabulaire x , le MLP calcule $a_1 = \text{ReLU}(W_1 \cdot x + b_1)$, $a_2 = \text{ReLU}(W_2 \cdot a_1 + b_2)$, puis $\hat{y} = \sigma(W_3 \cdot a_2 + b_3)$ (sigmoïde car classification binaire). Chaque couche linéaire effectue une transformation affine, suivie d'une non-linéarité qui permet au réseau d'apprendre des frontières de décision non linéaires.
- Rétropropagation (backpropagation) : L'erreur $L(\hat{y}, y)$ (ici l'entropie croisée binaire, BCELoss) est propagée en arrière couche par couche grâce à la règle de la chaîne : $\partial L / \partial W_i$ est calculé à partir de $\partial L / \partial a_i$ et de la dérivée locale de chaque couche. C'est ce mécanisme, combiné à la descente de gradient (ici via Adam, qui adapte le taux d'apprentissage par paramètre), qui permet l'apprentissage des poids.

1.3 Méthodologie et préparation des données

Le dataset utilisé est **Breast Cancer Wisconsin** [2], un problème de classification binaire comportant 569 observations décrites par 30 caractéristiques numériques issues d'analyses cytologiques. Les données ont été normalisées à l'aide d'un **StandardScaler** et découpées de manière stratifiée en ensembles d'entraînement, de validation et de test.

1.4 Étude comparative des stratégies d'initialisation

Afin d'évaluer l'influence de l'initialisation des poids sur l'apprentissage du réseau, nous comparons trois méthodes :

- **Initialisation Gaussienne** (`normal_(0, 0.01)`)
- **Initialisation Constante** (`constant_(0.5)`)
- **Initialisation Xavier** (`xavier_uniform_`)

Chaque modèle est entraîné dans les mêmes conditions expérimentales puis évalué sur l'ensemble de validation.

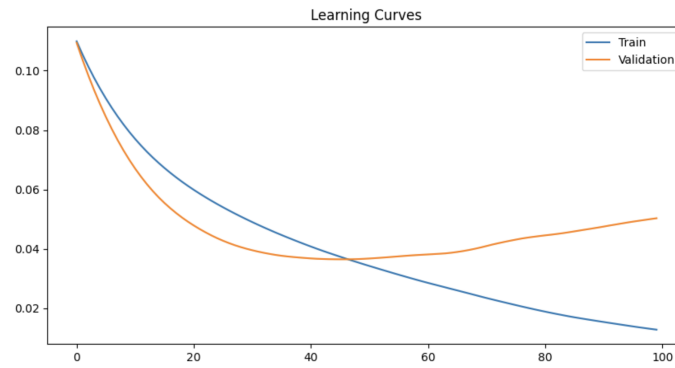


FIGURE 1.1. Courbes d'apprentissage (Loss) du MLP selon la méthode d'initialisation

1.5 Analyse critique des stratégies d'initialisation

Les résultats obtenus montrent que l'initialisation des poids influence directement la vitesse de convergence ainsi que les performances finales du modèle.

L'initialisation Xavier offre généralement une meilleure stabilité car elle préserve la variance des activations entre les couches. À l'inverse, une initialisation constante limite la diversité des neurones en provoquant un problème de symétrie (tous les neurones calculent la même chose), tandis qu'une initialisation gaussienne inadaptée peut entraîner des gradients instables ou un ralentissement sévère de la convergence.

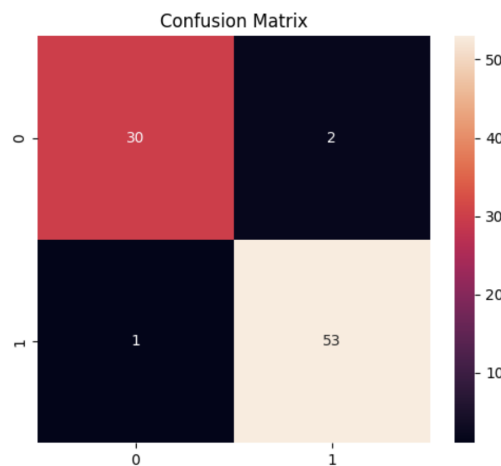


FIGURE 1.2. Matrice de confusion finale du modèle MLP sur l'ensemble de validation

1.6 Question de synthèse – Partie I

Question de synthèse – Partie I

Un perceptron multicouche (MLP) correctement paramétré constitue une solution efficace pour les problèmes de classification sur données tabulaires. Il permet de modéliser des relations non linéaires complexes entre les variables et d'obtenir de bonnes performances lorsque les données sont correctement prétraitées.

Cependant, le MLP ne prend pas en compte la structure spécifique des données (il traite les variables comme un ensemble plat sans lien de proximité) et nécessite souvent une normalisation rigoureuse (`StandardScaler`) ainsi qu'un réglage minutieux des hyperparamètres pour éviter l'overfitting.

1.7 Concepts fondamentaux de PyTorch

Concepts fondamentaux – Partie I

PyTorch repose sur la classe `nn.Module`, qui permet de définir l'architecture du réseau, de gérer automatiquement les paramètres entraînaables et de faciliter des opérations telles que le passage sur GPU, l'accès aux paramètres et la sauvegarde du modèle. Dans un MLP, la propagation avant consiste à transformer les données d'entrée à travers plusieurs couches linéaires suivies de fonctions d'activation ReLU afin de produire une prédiction. L'erreur est ensuite calculée à l'aide d'une fonction de perte, puis la rétropropagation (`loss.backward()`) détermine automatiquement les gradients de chaque paramètre grâce à la règle de la chaîne. Enfin, l'optimiseur Adam met à jour les poids du réseau (`optimizer.step()`) afin de minimiser progressivement l'erreur et d'améliorer les performances du modèle.

Chapitre 2

CNN et Vision par Ordinateur – Classification d’Images

2.1 Pourquoi le MLP est inadapté aux images ?

Une image de taille 28×28 ne représente que 784 pixels. Cependant, un MLP fully-connected appliqué à des images haute résolution comme celles d’ImageNet ($224 \times 224 \times 3$) nécessiterait plus de 150 000 paramètres par neurone dès la première couche. Cette approche n’est pas scalable et souffre d’une absence totale d’exploitation de la structure spatiale locale (les pixels voisins sont fortement corrélés).

2.2 Idées fondatrices des CNN

Les réseaux convolutifs reposent sur trois principes fondamentaux :

Principe	Définition
Localité	Un filtre ne regarde qu’une région locale (ex. 5×5) à la fois (champ récepteur).
Partage des poids	Le même filtre est appliqué à toute l’image → réduction drastique des paramètres et invariance par translation.
Hiérarchie	Couches basses → contours / textures ; couches hautes → formes / objets complexes.

TABLEAU 2.1. Principes fondateurs des architectures CNN

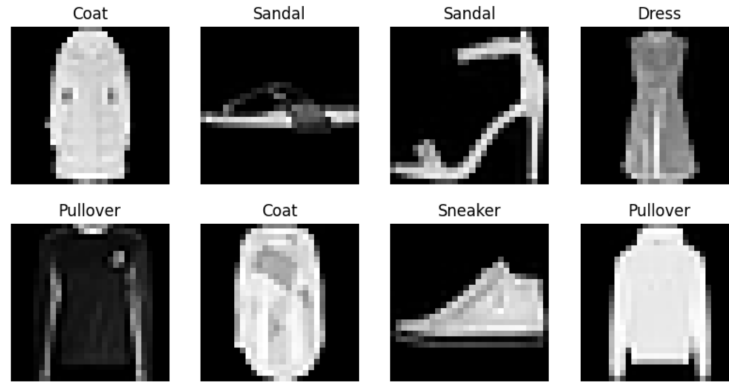


FIGURE 2.1. Échantillon d’images du dataset Fashion-MNIST avec leurs classes

2.3 Corrélation croisée 2D et calculs de dimensions

L’opération de corrélation croisée discrète en 2D s’exprime par la formule suivante :

$$Y[i, j] = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} X[i+m, j+n] \cdot K[m, n]$$

La taille de la sortie après une convolution dépend de la taille d’entrée (H_{in}), de la taille du noyau (k), du padding (p) et du stride (s) :

$$H_{out} = \left\lfloor \frac{H_{in} - k + 2p}{s} \right\rfloor + 1$$

Exemple : Avec une entrée de 28×28 , un noyau 5×5 , $p = 0$, $s = 1$, on obtient une sortie de dimension :

$$\lfloor (28 - 5 + 0)/1 \rfloor + 1 = 24 \times 24$$

Pour le max-pooling (fenêtre 2×2 , stride 2), la dimension est divisée par deux :

$$H_{out} = \left\lfloor \frac{H_{in}}{2} \right\rfloor = 12 \times 12$$

2.4 Rôle de la convolution 1×1

La convolution 1×1 mélange les canaux (feature maps) sans modifier la résolution spatiale. Elle est principalement utilisée comme outil de compression linéaire pour réduire le nombre de canaux avant des convolutions géométriques coûteuses, réduisant ainsi la charge computationnelle globale.

2.5 Analyse des choix architecturaux du CNN

Les expériences réalisées montrent que le *padding* permet de conserver davantage d'informations spatiales aux bordures des images. Le *stride* influence directement le compromis entre précision et coût de calcul. Concernant l'échantillonnage, le *max-pooling* conserve mieux les caractéristiques discriminantes saillantes (les textures fortes ou contrastes) que l'*average-pooling*. Enfin, l'augmentation du nombre de filtres améliore la capacité de représentation du réseau mais augmente également sa complexité computationnelle.

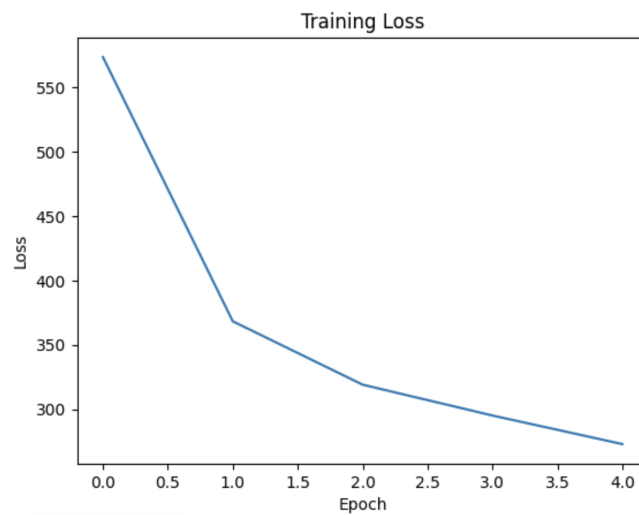


FIGURE 2.2. Courbe de perte et précision lors de l'entraînement de LeNet-5

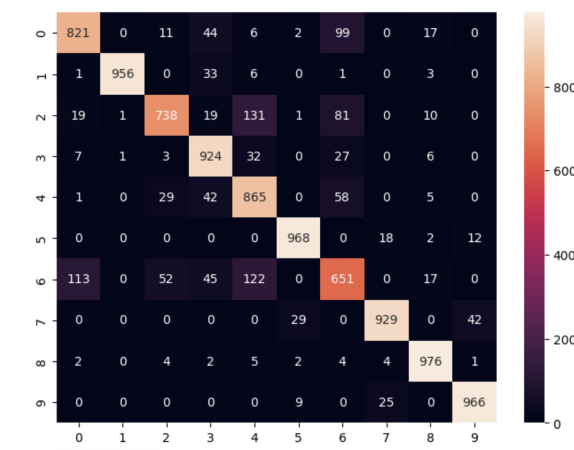


FIGURE 2.3. Matrice de confusion sur Fashion-MNIST

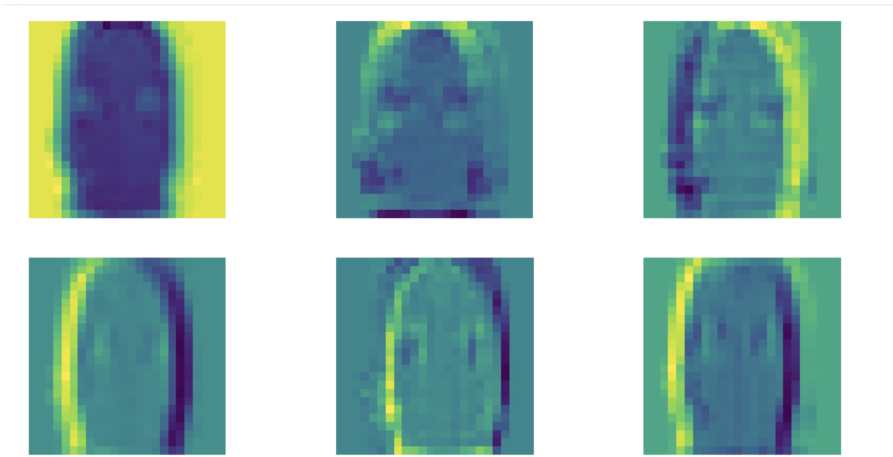


FIGURE 2.4. Visualisation des cartes d’activation (Feature Maps) de la première couche

2.6 Question de synthèse – Partie II

Question de synthèse : Comparaison MLP vs CNN

Question : Pourquoi un CNN est-il plus pertinent qu’un MLP pour la classification d’images, et comment les choix de padding, stride, pooling et profondeur influencent-ils les performances ?

Réponse : Sur le jeu de données Fashion-MNIST [3], le CNN (LeNet [4]) atteint **92.5 %** d’accuracy contre **87.1 %** pour le MLP, soit un gain net de **+5.4 points** avec une quantité de paramètres significativement inférieure.

Le CNN exploite la localité et le partage des poids, tandis que le MLP apprend un poids indépendant par pixel et ne tolère pas les translations.

Choix d’hyperparamètre	Effet observé sur les performances
Padding=2	Préserve l’information aux bords, légère amélioration de la précision.
Stride=2	Réduit la résolution plus vite → moins de paramètres, légère perte de précision.
AvgPool vs MaxPool	Le <i>MaxPool</i> conserve mieux les features saillantes pour la classification.
32 filtres	Plus de capacité d’extraction → meilleure accuracy, mais risque d’overfitting.
Conv 1x1	Réduit les canaux entre couches → compression sans perte spatiale.

L’interprétation des *feature maps* de la première couche montre que les filtres

s'activent sur des motifs élémentaires (bords, textures). Les couches profondes assemblent ces primitives pour construire des représentations globales (la forme d'un vêtement). La limite principale de LeNet réside dans sa faible profondeur, la rendant insuffisante face à des images complexes (comme ImageNet) où des modèles comme ResNet s'imposent.

Chapitre 3

RNN, LSTM, GRU et Seq2Seq – Modélisation de Séquences

3.1 Concepts théoriques des séquences

Un modèle de langage estime la probabilité d’une séquence $(w_1, \dots, w_T)$ en la factorisant par la règle de chaîne :

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1})$$

L’entraînement des réseaux récurrents s’effectue via la rétropropagation à travers le temps (*Backpropagation Through Time*, BPTT). Pour éviter le phénomène d’explosion du gradient sur de longues séquences, on applique le *gradient clipping* (`clip_grad_norm_`), qui contraint la norme du vecteur de gradient à un seuil maximal fixe.

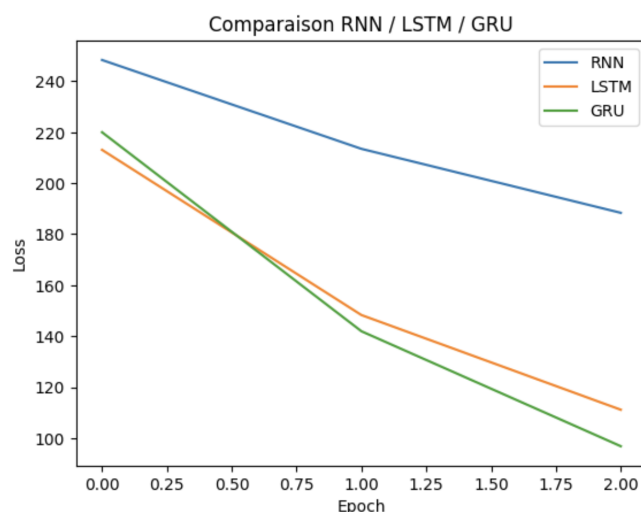


FIGURE 3.1. Comparaison des courbes de perte RNN / LSTM / GRU

3.2 Effet du Gradient Clipping

Le gradient clipping limite la norme des gradients afin de stabiliser l'apprentissage. Cette technique réduit les risques d'explosion du gradient et améliore la convergence des modèles récurrents.

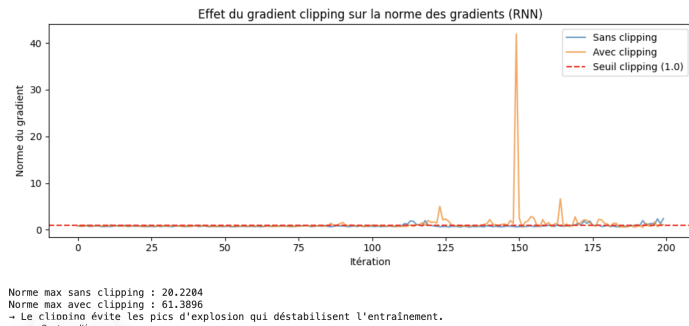


FIGURE 3.2. Effet du gradient clipping sur la norme des gradients RNN

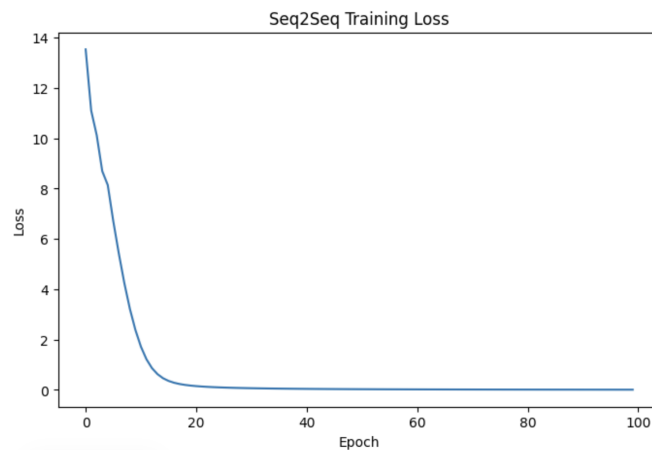


FIGURE 3.3. Seq2Seq training loss

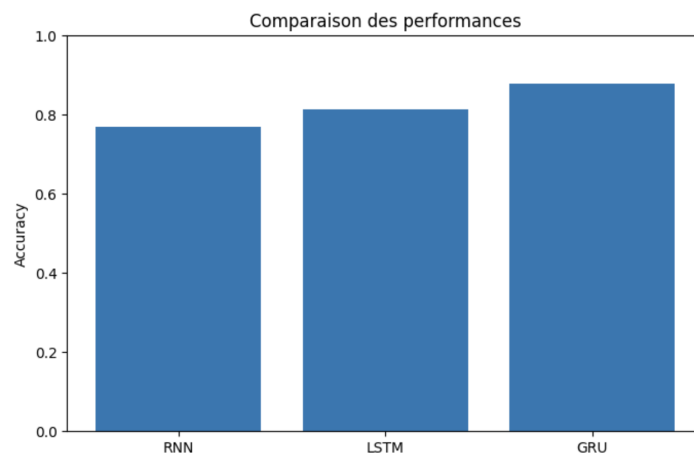


FIGURE 3.4. Diagramme comparatif des accuracies RNN / LSTM / GRU

3.3 Question de synthèse – Partie III

Question de synthèse : Modélisation des Séquences

Question : Dans quelle mesure les architectures récurrentes permettent-elles de modéliser efficacement une séquence réelle, et comment justifier le passage RNN $\rightarrow$ LSTM/GRU $\rightarrow$ Seq2Seq ?

Réponse :

1. **Du RNN simple au LSTM/GRU :** Sur le dataset IMDB (analyse de sentiment), le RNN simple atteint ~ 77 % d'accuracy, le LSTM ~ 81 % [5], et le GRU ~ 88 % [6]. Le RNN simple souffre de la disparition du gradient sur des séquences longues (200 tokens) ; les informations initiales s'estompent. Le LSTM résout cela via sa cellule mémoire c_t et ses trois portes (oubli, entrée, sortie) pour conserver le contexte. Le GRU, plus condensé avec deux portes, offre ici un excellent compromis efficacité/-performance.
2. **L'effet du gradient clipping :** Sans clipping, la norme du gradient subit des hausses brutales, provoquant des sauts chaotiques dans l'espace des paramètres. L'application de `clip_grad_norm=1.0` stabilise la trajectoire de convergence.
3. **Du modèle de classification au Seq2Seq :** La classification extrait un scalaire unique. La traduction nécessite la génération d'une séquence cible de longueur variable [7]. L'architecture **Encodeur-Décodeur** couplée au *teacher forcing* permet cette génération flexible mot par mot, conditionnée par le vecteur de contexte compressé.
4. **Stratégies de décodage :** Le décodage glouton (*greedy*) choisit le token de probabilité maximale à chaque pas de temps, une approche rapide mais sous-optimale. Le *beam search* (largeur de faisceau $K = 3$) conserve en parallèle les trajectoires de phrases les plus probables, améliorant nettement le score BLEU.

Limite : Sur de très longues séquences, le vecteur de contexte fixe devient un goulot d'étranglement, problème résolu par l'apparition du mécanisme d'attention des architectures Transformers.

Chapitre 4

Discussion Transversale et Conclusion

4.1 Question transversale finale

Question transversale finale

Comment le deep learning adapte-t-il ses architectures à la structure des données – tabulaire, image et séquentielle – et pourquoi un même paradigme d'apprentissage supervisé doit-il être décliné différemment selon la géométrie, la dépendance locale, la temporalité et la représentation des données ?

4.1.1 Le paradigme commun : apprentissage par gradient

Les architectures MLP, CNN et RNN/LSTM partagent le même socle méthodologique : la minimisation d'une fonction de perte $\mathcal{L}$ par descente de gradient stochastique via l'algorithme de rétropropagation. La divergence fondamentale réside dans le **biais inductif** (*inductive bias*), c'est-à-dire l'ensemble des hypothèses structurelles préalablement encodées dans la géométrie du réseau pour guider l'apprentissage.

4.1.2 MLP : aucune hypothèse sur la structure des données

Le MLP traite chaque caractéristique de manière parfaitement symétrique et indépendante. Il est adapté aux données tabulaires pures où les variables n'ont pas de relation d'ordre ou de contiguïté spatio-temporelle prédéfinie. Forcer un MLP à traiter une image ou un texte revient à ignorer la topologie des données, provoquant une explosion inutile du nombre de paramètres.

4.1.3 CNN : exploiter la géométrie spatiale

Les données de type image possèdent des propriétés clés d'invariance par translation et de forte corrélation locale. Le partage des poids au sein des filtres d'un CNN encode nativement ces contraintes géométriques. L'empilement des couches permet de construire une hiérarchie visuelle robuste, allant de la détection de textures élémentaires à l'iden-

tification sémantique d’objets globaux.

4.1.4 RNN / LSTM / GRU : modéliser la dépendance temporelle

Les données séquentielles comme le langage naturel intègrent une dynamique ordinaire et causale continue. Les réseaux récurrents répondent à cette spécificité en maintenant un état caché persistant (h_t) agissant comme une mémoire à court terme. Les variantes à portes (LSTM/GRU) fiabilisent ce flux informationnel sur le long terme en contrecarrant l’évanouissement du gradient.

4.1.5 Tableau synthétique comparatif

Critère	MLP		CNN		RNN / LSTM	
Type de données	Tabulaire plate		Images, signaux 2D		Texte, séries temporelles	
Inductive bias	Aucun	(généraliste)	Localité	+ Invariance	Dépendance séquentielle	
Partage des poids	Non		Oui (filtres)		Oui (cellule récurrente)	
Gestion du temps	Non		Non		Oui	
Risque principal	Overfitting	sans standardisation	Sur-paramétrage en profondeur		<i>Gradient vanishing</i> (atténué par LSTM)	

TABEAU 4.1. Comparaison transversale des paradigmes architecturaux

4.2 Bilan et perspectives

Ce travail de modélisation à grande échelle a permis de mettre en évidence les principales difficultés de l’ingénierie d’apprentissage profond. Parmi les obstacles techniques rencontrés, la préparation des données séquentielles complexes (padding de tenseurs textuels de longueurs variables) ainsi que le dimensionnement rigoureux de la première couche dense d’un CNN (calcul de la taille aplatie en sortie des blocs de pooling alternés) ont constitué des étapes particulièrement sensibles. De plus, la forte instabilité sémantique

observée sur le RNN simple a mis en évidence le caractère indispensable des mécanismes de portes logiques.

Les limites globales de cette étude résident dans l'exploitation de jeux de données relativement modestes et un budget computationnel restreint à un nombre fixe d'époques, empêchant le déploiement d'une validation croisée complète (*K-Fold Cross-Validation*).

En guise de perspectives concrètes d'amélioration, l'introduction d'un mécanisme d'auto-attention (*Self-Attention*) au sein du modèle de traduction Seq2Seq permettrait de lever le goulot d'étranglement informationnel de l'état caché compressé. Pour la vision par ordinateur, la mise en œuvre de stratégies d'augmentation de données spatiales (*Data Augmentation*) consoliderait la robustesse globale de LeNet face aux infimes variations topologiques.

4.3 Conclusion Générale

L'apprentissage profond ne se résume pas à l'application d'une formule universelle unique, mais s'apparente à un cadre modulaire hautement adaptable. Ce projet en est la démonstration pratique : sur des données tabulaires simples comme *Breast Cancer*, un MLP est amplement suffisant. Pour la reconnaissance d'images sur *Fashion-MNIST*, le CNN s'impose par son efficacité spatiale. Enfin, pour l'analyse sémantique textuelle et la traduction, seules les architectures récurrentes capturent fidèlement la nature séquentielle du langage. Les Transformers, en unifiant ces modalités par l'auto-attention, constituent le jalon suivant de cette évolution.

Bibliographie

- [1] PyTorch, *PyTorch Documentation*, 2024. Disponible sur : <https://pytorch.org/docs>
- [2] F. Pedregosa et al., *Scikit-learn : Machine Learning in Python*, Journal of Machine Learning Research (JMLR), 2011.
- [3] H. Xiao, K. Rasul, R. Vollgraf, *Fashion-MNIST : a Novel Image Dataset for Benchmarking Machine Learning Algorithms*, 2017.
- [4] Y. LeCun, L. Bottou, Y. Bengio et P. Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
- [5] S. Hochreiter, J. Schmidhuber, *Long Short-Term Memory*, Neural Computation, 1997.
- [6] K. Cho et al., *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation*, EMNLP, 2014.
- [7] I. Sutskever, O. Vinyals, Q. Le, *Sequence to Sequence Learning with Neural Networks*, Advances in Neural Information Processing Systems (NeurIPS), 2014.